

Open in app ↗

≡ Medium



Devops & AI Hub · Following

★ Member-only story

DevOps



Agentic Ai



Architecture



Cloud Computing



AI



Agentic DevOps With MCP: A Real Production Architecture

8 min read · Jun 15, 2026



Neel Shah

Follow

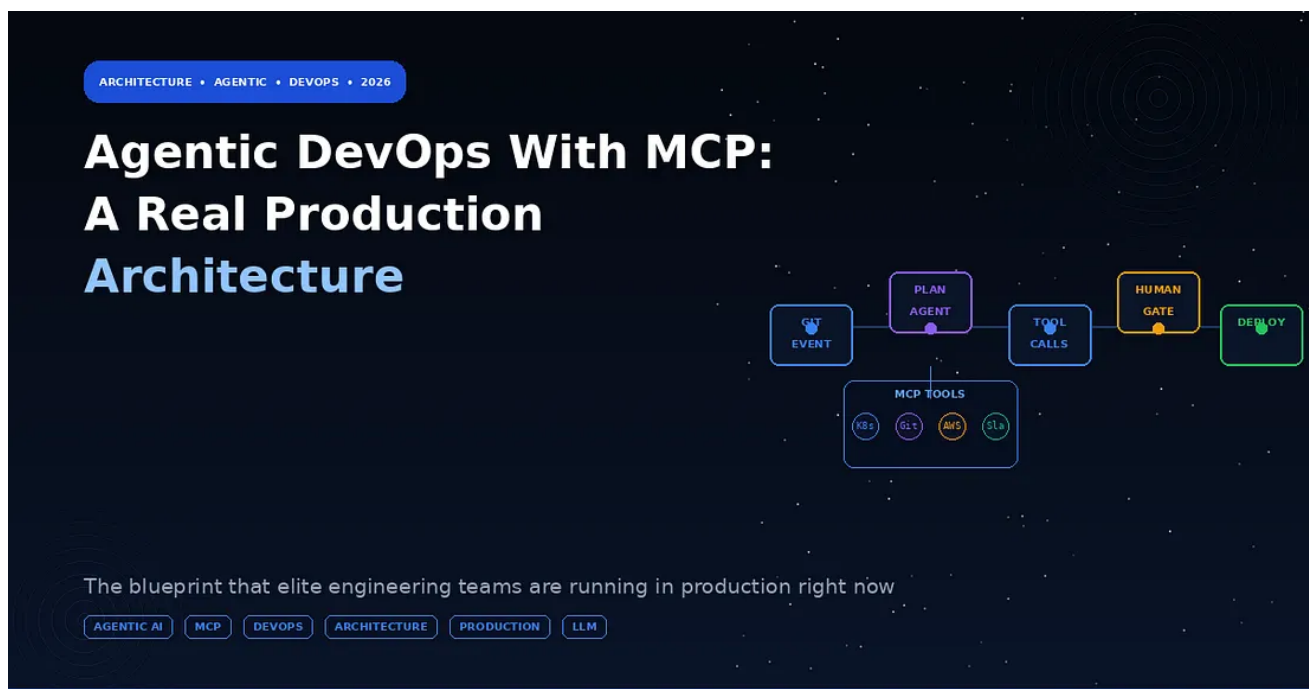


Listen



Share

... More



Most writing about agentic DevOps describes what is theoretically possible. This guide describes what is actually running in production at engineering organisations that have moved past experimentation. The gap between

‘impressive demo’ and ‘production AI agent’ is significant — and the decisions made in bridging that gap define whether agentic DevOps delivers ROI or becomes an abandoned innovation project.

A production agentic DevOps architecture is not a single AI model connected to a set of tools. It is a system of components: an orchestration layer that manages the agent loop, an MCP tool layer that provides safe, audited access to infrastructure, an approval layer that intercepts high-risk actions, a memory layer that maintains context across tasks, and an observability layer that makes the system auditable and debuggable. Each component matters; removing any one of them degrades the system from ‘production-grade’ to ‘prototype’.

This guide documents the architecture that has emerged across multiple production deployments in 2026 — the decisions, trade-offs, and implementation details that make the difference between an AI agent that actually helps engineers and one that creates new operational risk.

ARCHITECTURE SCOPE

This guide covers a complete agentic DevOps architecture: orchestration layer, MCP tool layer, approval gates, memory/context management, observability, and the operational model for running it in production. Code examples are production-tested Python and YAML.

The Production Architecture: Overview

Layer	Component	Role	Technology
Event Layer	Alert/trigger ingestion	Receives signals from CI/CD, monitoring, Git	PagerDuty webhooks, GitHub webhooks, cron
Orchestration	Agent loop manager	Runs the ReAct loop; plans, acts, observes	LangGraph, Anthropic Agent SDK
LLM Layer	Reasoning engine	Generates plans, interprets results, makes decisions	Claude 3.7 Sonnet / GPT-4o
Tool Layer	MCP servers	Executes actions against real systems	Kubernetes MCP, GitHub MCP, AWS MCP
Approval Layer	Human-in-the-loop gate	Intercepts write ops above risk threshold	Slack approval flows, PagerDuty webhooks
Memory Layer	Context store	Persists knowledge across agent runs	Chroma/pgvector + structured state DB
Audit Layer	Action logger	Records every action for accountability	OpenTelemetry + structured JSON logs
Observability	Agent monitoring	Monitors agent health and decision quality	LangSmith, Langfuse, Grafana

Layer 1: The Orchestration Core

The orchestration layer is the heart of the agentic system — it manages the ReAct (Reason-Act-Observe) loop that lets the agent plan multi-step workflows, execute tools, observe results, and plan the next step based on what it learned.

```
# orchestration/agent.py – LangGraph-based agent with MCP tools
from langchain_anthropic import ChatAnthropic
from langchain_mcp import MCPToolkit
from langgraph.prebuilt import create_react_agent
from langgraph.checkpoint.postgres import PostgresSaver
from approval import ApprovalGate
import logging

log = logging.getLogger('devops-agent')

# — System prompt: defines agent identity and constraints —
SYSTEM_PROMPT = '''
You are a senior DevOps agent with access to Kubernetes, GitHub, AWS, and Da

CORE RULES (never violate these):
1. Before taking any write action (apply, delete, modify, merge), call request_approval with a description of what you plan to do and why.
2. Never take multiple write actions in sequence without approval for each.
```

3. If you are uncertain about the root cause (confidence < 80%), investigate before proposing remediation.
4. Always check recent deployments before concluding a non-deployment cause.
5. Log every significant finding with `log_finding()` before acting on it.

INVESTIGATION ORDER for incidents:

Step 1: Scope – which service, what impact, since when?

Step 2: Recent changes – deployments, config changes, infrastructure changes

Step 3: Current state – pod status, resource utilisation, error rates

Step 4: Logs – structured error patterns, stack traces

Step 5: Root cause – form hypothesis with confidence level

Step 6: Propose remediation – never auto-execute without approval

'''

— MCP Tool Registration —————

```
def build_agent(approval_gate: ApprovalGate):
    # Load MCP toolkits
    k8s_tools = MCPToolkit(server='kubernetes-mcp').get_tools()
    gh_tools = MCPToolkit(server='github-mcp').get_tools()
    aws_tools = MCPToolkit(server='aws-mcp').get_tools()
    dd_tools = MCPToolkit(server='datadog-mcp').get_tools()

    # Wrap write tools with approval gate
    safe_tools = []
    for tool in k8s_tools + gh_tools + aws_tools:
        if tool.name in WRITE_TOOL_NAMES:
            safe_tools.append(approval_gate.wrap(tool))
        else:
            safe_tools.append(tool)

    # Add audit tools
    all_tools = safe_tools + dd_tools + [approval_gate.request_tool]

    # Persistent memory – agent remembers context across runs
    memory = PostgresSaver.from_conn_string(os.environ['DATABASE_URL'])

    model = ChatAnthropic(model='claude-3-7-sonnet-20250219', max_tokens=4096)

    return create_react_agent(
        model=model,
        tools=all_tools,
        checkpointer=memory,
        state_modifier=SYSTEM_PROMPT,
    )

WRITE_TOOL_NAMES = {
    'apply_manifest', 'patch_resource', 'delete_resource',
    'create_pull_request', 'merge_pull_request',
    'create_issue', 'terminate_instance',
}
```

Layer 2: The Approval Gate — The Critical Safety Mechanism

The approval gate is what separates a production AI agent from a dangerous one. It intercepts every write operation, formats a human-readable description of the proposed action, sends it to Slack (or PagerDuty), and waits for explicit approval before allowing execution. Without this, AI write operations execute immediately and unconditionally — unacceptable for production infrastructure.

```
# approval/gate.py – Slack-based human approval for write operations
from slack_sdk import WebClient
from slack_sdk.errors import SlackApiError
import time, json, uuid
from langchain.tools import BaseTool
from pydantic import BaseModel

class ApprovalGate:
    def __init__(self, slack_token: str, channel: str, timeout_seconds: int):
        self.slack = WebClient(token=slack_token)
        self.channel = channel
        self.timeout = timeout_seconds
        self.pending: dict = {}

    def wrap(self, tool: BaseTool) -> BaseTool:
        '''Wrap a write tool with approval gate.'''
        original_invoke = tool._run

        def guarded_run(*args, **kwargs):
            approval_id = str(uuid.uuid4())[:8]
            description = f'**{tool.name}** called with:\n```\n{json.dumps(kwargs)}\n```'

            approved = self._request_approval(
                approval_id=approval_id,
                tool_name=tool.name,
                description=description,
                risk_level=self._assess_risk(tool.name, kwargs),
            )

            if not approved:
                return f'Action {tool.name} was rejected by human reviewer ('

            result = original_invoke(*args, **kwargs)
            self._log_approved_action(approval_id, tool.name, kwargs, result)
            return result

        tool._run = guarded_run
        return tool
```

```

def _assess_risk(self, tool_name: str, args: dict) -> str:
    HIGH_RISK = {'delete_resource', 'terminate_instance', 'merge_pull_re
    MEDIUM_RISK = {'apply_manifest', 'patch_resource', 'create_pull_requ
    if tool_name in HIGH_RISK: return 'HIGH'
    if tool_name in MEDIUM_RISK: return 'MEDIUM'
    return 'LOW'

def _request_approval(self, approval_id, tool_name, description, risk_level):
    risk_emoji = {'HIGH': ':red_circle:', 'MEDIUM': ':yellow_circle:', 'LOW': ':green_circle:'}
    msg = self.slack.chat_postMessage(
        channel=self.channel,
        blocks=[
            {'type': 'section', 'text': {'type': 'mrkdwn',
            'text': f'{risk_emoji[risk_level]} *AI Agent Action Request: {description}*'},
            {'type': 'actions', 'elements': [
                {'type': 'button', 'text': {'type': 'plain_text', 'text': 'Approve',
                'style': 'primary', 'value': f'approve:{approval_id}'},
                {'type': 'button', 'text': {'type': 'plain_text', 'text': 'Reject',
                'style': 'danger', 'value': f'reject:{approval_id}'}, 'a
            ]},
        ]
    )
    return self._wait_for_response(approval_id)

def _wait_for_response(self, approval_id: str) -> bool:
    deadline = time.time() + self.timeout
    while time.time() < deadline:
        if approval_id in self.pending:
            return self.pending.pop(approval_id)
        time.sleep(2)
    return False # Timeout = rejection

```

Layer 3: Event-Driven Agent Triggering

A production agentic DevOps system responds to events automatically, not just when a human asks a question. The three primary triggers: alerts from monitoring systems, webhooks from CI/CD pipelines, and scheduled maintenance tasks.

```

# triggers/webhook_server.py – FastAPI server for event-driven triggers
from fastapi import FastAPI, BackgroundTasks
from pydantic import BaseModel
from orchestration.agent import build_agent
import asyncio

```

```

app = FastAPI()
agent = build_agent(approval_gate)

class PagerDutyWebhook(BaseModel):
    event: str
    incident: dict

class GitHubWebhook(BaseModel):
    action: str
    pull_request: dict | None = None
    deployment: dict | None = None

@app.post('/webhooks/pagerduty')
async def handle_pagerduty(webhook: PagerDutyWebhook, bg: BackgroundTasks):
    if webhook.event == 'incident.trigger':
        incident = webhook.incident
        goal = f'''
        PagerDuty incident triggered:
        Service: {incident['service']['name']}
        Title: {incident['title']}
        Urgency: {incident['urgency']}
        Triggered at: {incident['created_at']}

        Investigate this incident comprehensively:
        1. Identify the affected service and check its current health
        2. Check recent deployments in the last 2 hours
        3. Examine relevant logs and metrics
        4. Form a root cause hypothesis with confidence level
        5. Post your findings to Slack #incidents with a structured summary
        6. Only propose remediation if confidence > 85%
        '''

        bg.add_task(run_agent, goal, thread_id=incident['id'])
    return {'status': 'accepted'}

@app.post('/webhooks/github')
async def handle_github(webhook: GitHubWebhook, bg: BackgroundTasks):
    # Trigger agent for deployment events to monitor for rollout issues
    if webhook.action == 'completed' and webhook.deployment:
        dep = webhook.deployment
        goal = f'''
        A deployment just completed: {dep['environment']} / {dep['ref']}
        Monitor for the next 10 minutes:
        - Check error rates every 2 minutes
        - Watch for new CrashLoopBackoff or OOMKill events
        - Alert immediately if error rate exceeds baseline by 20%
        '''

        bg.add_task(run_agent, goal, thread_id=f'deploy-{dep["id"]}')
    return {'status': 'accepted'}

async def run_agent(goal: str, thread_id: str):
    config = {'configurable': {'thread_id': thread_id}}
    result = await agent.ainvoke({'messages': [('user', goal)]}, config=config)

```

```
return result
```

Layer 4: Memory and Context Persistence

A stateless AI agent that starts each task with no context from previous tasks is significantly less effective than one that accumulates knowledge over time. The memory layer provides two types of persistence: episodic memory (what happened in previous incidents and tasks) and semantic memory (knowledge about your specific infrastructure that the agent has learned).

```
# memory/knowledge_store.py – RAG-based infrastructure knowledge base
from langchain_community.vectorstores import Chroma
from langchain_anthropic import AnthropicEmbeddings
from langchain.schema import Document

class InfraKnowledgeStore:
    """
    Stores and retrieves infrastructure knowledge:
    - Postmortem summaries and resolutions
    - Service architecture documentation
    - Known failure patterns and their causes
    - Historical incident context
    """

    def __init__(self):
        self.embeddings = AnthropicEmbeddings()
        self.store = Chroma(
            collection_name='infra_knowledge',
            embedding_function=self.embeddings,
            persist_directory='/data/knowledge-store',
        )

    def add_postmortem(self, service: str, incident_id: str,
                      root_cause: str, resolution: str):
        doc = Document(
            page_content=f'Service: {service}\nRoot Cause: {root_cause}\nRes',
            metadata={'type': 'postmortem', 'service': service, 'incident_id': incident_id}
        )
        self.store.add_documents([doc])

    def query_similar_incidents(self, symptom_description: str, k: int = 3):
        docs = self.store.similarity_search(symptom_description, k=k)
        return [d.page_content for d in docs]

# Usage: inject historical context into agent task
```

```
knowledge = InfraKnowledgeStore()
similar = knowledge.query_similar_incidents(
    'payments-api high error rate after deployment'
)
# Returns: past postmortems with similar symptoms → agent uses these
# to form better initial hypotheses before even querying the cluster
```

Layer 5: Observability for the Agent Itself

AI agents are complex systems that can fail in subtle ways: they can get stuck in reasoning loops, make incorrect diagnoses with high confidence, call tools in inefficient sequences, or miss important information because their context window is full. You need observability for the agent itself — not just for the infrastructure it manages.

```
# observability/agent_tracer.py – OTel instrumentation for agent runs
from opentelemetry import trace
from opentelemetry.exporter.otlp.proto.grpc.trace_exporter import OTLPSpanExporter
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
import time, json

tracer_provider = TracerProvider()
tracer_provider.add_span_processor(
    BatchSpanProcessor(OTLPSpanExporter(endpoint='http://otel-collector:4317'))
)
tracer = tracer_provider.get_tracer('devops-agent')

def trace_agent_run(task_goal: str, thread_id: str):
    with tracer.start_as_current_span('agent.run') as span:
        span.set_attribute('agent.task_goal', task_goal[:500])
        span.set_attribute('agent.thread_id', thread_id)
        span.set_attribute('agent.model', 'claude-3-7-sonnet')

def trace_tool_call(tool_name: str, args: dict, result: str, duration_ms: float):
    with tracer.start_as_current_span('agent.tool_call') as span:
        span.set_attribute('tool.name', tool_name)
        span.set_attribute('tool.args', json.dumps(args)[:500])
        span.set_attribute('tool.result_length', len(result))
        span.set_attribute('tool.duration_ms', duration_ms)
        span.set_attribute('tool.is_write_op', tool_name in WRITE_TOOL_NAMES)

# Metrics to track in Grafana:
# - agent_run_duration_seconds (histogram)
# - agent_tool_calls_total (counter, by tool_name)
```

```
# - agent_approval_requests_total (counter, by outcome: approved/rejected/ti
# - agent_task_success_rate (gauge)
# - agent_tokens_used_total (counter, by model)
```

Deployment: Running the Agent in Kubernetes

```
# k8s/agent-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: devops-agent
  namespace: platform-tools
spec:
  replicas: 1 # Single instance – agent has stateful reasoning
  selector:
    matchLabels: { app: devops-agent }
  template:
    spec:
      serviceAccountName: devops-agent-sa
      containers:
        - name: agent
          image: ghcr.io/yourorg/devops-agent:latest
          env:
            - name: ANTHROPIC_API_KEY
              valueFrom:
                secretKeyRef: { name: ai-credentials, key: anthropic-api-key }
            - name: SLACK_BOT_TOKEN
              valueFrom:
                secretKeyRef: { name: slack-credentials, key: bot-token }
            - name: DATABASE_URL # For agent memory persistence
              valueFrom:
                secretKeyRef: { name: db-credentials, key: url }
          resources:
            requests: { cpu: 200m, memory: 512Mi }
            limits: { cpu: 1000m, memory: 2Gi }
---
# RBAC for agent – read-only cluster access
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: devops-agent-readonly
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: view # Standard read-only cluster role
subjects:
  - kind: ServiceAccount
```

```
name: devops-agent-sa
namespace: platform-tools
```

Production Results: What to Expect

Metric	Typical Result After 90 Days	Notes
P2/P3 alert autonomous resolution	20-40%	Increases as agent learns your environment
MTTI improvement	60-75% faster	AI investigation vs. manual multi-tool lookup
Approval rejection rate	15-25% of proposed actions	Healthy — means human review is active
False positive action proposals	< 8%	Track via post-approval review process
Agent token cost (Claude Sonnet)	\$40-120/month	Scales with alert volume and task complexity
On-call page rate reduction	25-40%	From autonomous resolution of routine alerts

THE HONEST ASSESSMENT

A production agentic DevOps system requires 4–8 weeks of setup and tuning. The first month has a higher false-proposal rate (15–25%) as the system learns your environment. By month 3, proposal accuracy typically exceeds 90% for incident types the agent has seen before. Novel incidents still require significant human involvement. The ROI is real — but it requires treating the agent like a new team member who needs onboarding, not a magic button.

DevOps

+

Agentic Ai

+

Architecture

+

Cloud Computing

+

AI

+



Follow

Written by Neel Shah

1.4K followers · 27 following

Developer Advocate at Stackgen || Open Source and Devops Guy || Building DevOps Communities
@googlecloud @hashicorp @cncf @docker



Following

Published in Devops & AI Hub

1K followers · Last published 5 hours ago

All things related to Devops,SRE,Cloud,etc.

